

PacketCGAN: Exploratory Study of Class Imbalance for Encrypted Traffic Classification Using CGAN

Pan Wang*, Shuhang Li*, Feng Ye[†], Zixuan Wang* and Moxuan Zhang[‡]

*School of Modern Posts, Nanjing University of Posts & Telecommunications, Nanjing, China

[†]Department of Electrical & Computer Engineering, University of Dayton, Dayton, OH, USA

[‡]Schools of International Education, Jinling Institute of Technology, Nanjing, China

Email: *wangpan@njupt.edu.cn

Abstract—With the popularity of Deep Learning (DL), researchers have begun to apply DL to tackle with encrypted traffic classification problems. Although these methods can automatically extract traffic features to improve the ability of feature engineering of traditional methods like DPI, a large amount of data is still needed to learn the characteristics of various types of traffic. Therefore, the performance of classification model always significantly depends on the quality of datasets. Nonetheless, the building of datasets is a time-consuming and costly task, especially encrypted traffic. Apparently, it is often more difficult to collect a large amount of traffic samples of those unpopular applications than well-known ones, which often leads to the problem of class imbalance between major and minor encrypted applications in datasets. In this paper, we proposed a novel traffic data augmenting method called PacketCGAN using Conditional GAN, which can control the modes of data to be generated. PacketCGAN exploit the benefit of CGAN to generate specified samples with the input of applications' types as conditional and thereby achieve data balancing. As a proof of concept, three classical DL models including CNN were adopted to classify four types of encrypted traffic datasets augmented by Random Over Sampling (ROS), SMOTE(Synthetic Minority Over-sampling Technique), vanilla GAN and PacketCGAN respectively using public datasets. The experimental evaluation results demonstrate that DL based encrypted traffic classifier over our new dataset augmented by PacketCGAN can achieve better performance than the other three in terms of encrypted traffic classification.

Index Terms—encrypted traffic classification, data augmentation, Conditional Generative Adversarial Network, traffic identification, class imbalance

I. INTRODUCTION

With the rapid development of network technology, the types and quantity of traffic data in cyberspace are increasing. Network traffic identification and classification is a crucial research task in the area of network management and security. It is the footstone of dynamic access control, network resources scheduling, content based billing, intrusion and malware detection etc. High efficient and accurate traffic classification is of great practical significance to provide service quality assurance, dynamic access control and abnormal network behaviors detection. With the widespread adoption of encryption techniques for internet, especially 5G and IoT applications, the growth of portion of encrypted traffic has dramatically posed

a huge challenge for QoS, network management and security monitoring. Therefore, studies on encrypted traffic classification not only help to improve the fine-grained network resource allocation based on application, but also enhance security level of network and applications.

Traditionally, the evolution of encrypted traffic classification technology has gone through three stages: port matching, payload matching and flow statistical characteristics based classification methods. Port matching based method infers applications' types by assuming that most applications consistently use 'well known' TCP or UDP port numbers, however, the emergence of port camouflage, dynamic port, proprietary protocols with user-defined ports and tunneling technology makes these methods lose efficacy quickly. Payload matching based methods, namely, DPI (Deep Packet Inspection) technology cannot deal with encrypted traffic because of invisible packet content of encrypted traffic, in addition, it incurs high computational overhead and requires manual signatures maintenance [1, 2]. As a result, in order to attempt to solve the aboved problems of encrypted traffic identification, flow-based methods emerged, which usually combine statistical or time series traffic features with Machine Learning (ML) algorithms, such as naive bayes(NB), support vector machine(SVM), decision tree, Random Forest(RF), k-nearest neighbor(KNN) [3]. Although classical machine learning approaches can solve many issues that port and payload based methods cannot solve, it still has some limitations, such as handcrafted traffic features driven by domain-expert, time-consuming, lack of ability of automation, rapidly outdated when compared to the evolution. Unlike most traditional ML algorithms, DL performs automatic feature extraction without human intervention, which undoubtedly makes it a highly desirable approach for traffic classification, especially encrypted traffic. Recent research work has demonstrated the superiority of DL methods in traffic classification [4], such as MLP [5], CNN [6], SAE [7], LSTM [8].

However, due to the different popularity of various applications, the class imbalance problem of traffic samples often occurs when building traffic datasets. That is, the number of popular applications samples is much larger than others,

which always leads to the misclassifying problems of minor applications and thereby incurs deterioration of classifier performance. Imbalanced class distribution of a dataset has posed a serious challenge to most ML based classifiers which assume a relatively balanced distribution. Network traffic classification is no exception due to the imbalanced property of network traffic data [9], especially encrypted traffic. Therefore, it is very crucial to address such challenges of imbalanced class distribution of traffic datasets for network traffic classification. However, there are very few studies focusing on traffic data augmentation for traffic classification to overcome the limitation of class imbalance.

In this paper, we proposed traffic data augmentation method called PacketCGAN using Conditional GAN, one of a genre of GAN, which can control the modes of the data to be generated. As a generative model, PacketCGAN exploits the benefit of CGAN to generate synthesized traffic samples by learning the characteristics of the original traffic data. The synthesized data is then combined with the original (viz. real) data to build the new traffic dataset and thereby keep balance between major and minor classes of the dataset. As a proof of concept, three classical DL models like CNN were adopted and designed to classify four types of encrypted traffic datasets augmented by ROS, SMOTE, vanilla GAN and PacketCGAN respectively using public datasets. The experimental evaluation results demonstrate that DL based encrypted traffic classifier over our new dataset augmented by PacketCGAN can achieve better performance than the other three in terms of encrypted traffic classification.

The rest of this paper is organized as follows. Section II introduces related works of traffic classification and some current methods for tackling with the problem of imbalanced class data. Section III describes the principles of GAN and CGAN. Section IV illustrates the methodology of PacketCGAN. The experimental results are provided and discussed in Section V. Section VI concludes our work and presents some future works.

II. RELATED WORKS

A. DL based approach of Traffic Classification

Unlike most traditional ML algorithms, Deep Learning performs automatic feature extraction without human intervention, which undoubtedly makes it a highly desirable approach for traffic classification, especially encrypted traffic. Recent research work has demonstrated the superiority of DL methods in traffic classification [4–6, 8, 10]. The workflow of DL based classification usually consists of three steps. First, model inputs are defined and designed according to some principles, such as raw packets, PCAP files or flow statistics features. Second, models and algorithms are elaborately chosen according to models' characteristics and aim of the classifier. Finally, the DL classifier is trained to automatically extract the features of traffic.

B. Traditional methods for handling imbalanced data

In general, there are three methods for dealing with class imbalance problem: **Modifying the objective cost function**, **Sampling and Generating artificial data** [11]. The approach of **modifying objective cost function** alleviates the problem by means of weighting differently the data samples in minor and major classes, which gives higher score on the minor samples to penalize more intensely on miss-classifying of the sample in the minor class. Sampling methods include two different ways of **under-sampling** and **over-sampling**, which is to reduce the size of major class by removing some major data samples and raise the ones in the minor class, respectively. Random under sampling (RUS) and Random over sampling (ROS) are two main methods of under-sampling and over-sampling [12]. RUS randomly removes some instances in major class, accordingly, ROS generates some copies of samples of the minor class. However, overfitting problem is always the main drawback of ROS due to generating same copies from the minor class. A classical method for generating artificial data is Synthetic Minority Over-sampling Technique (SMOTE) in which minority samples are generated by synthetic samples rather than copies [13].

C. The application of GAN and other DL techniques in generating traffic data samples

Due to the great success of GAN applying in images, computer vision and NLP etc., this innovative technique has been already applied to network security recently. As for traffic classification, some researchers have introduced some approaches based on GAN to generate the traffic samples to overcome the imbalanced limitation of network data. In [14], the authors proposed a novel method called auxiliary classifier GAN (AC-GAN) to generate synthesized traffic samples for balancing between the minor and major classes over a well-known traffic dataset NIMS. The AC-GAN took both a random noise and a class label as input in order to generate the samples of the input class label accordingly. The experimental results has shown that their proposed method achieved better performance compared to other methods like SMOTE. However, the NIMS dataset was only composed of SSH and non-SSH two classes. In [15], the authors proposed a novel data augmentation approach based on the use of Long Short Term Memory (LSTM) network to learn the traffic flow patterns and Kernel Density Estimation (KDE) for replicating the sequence of packets in a flow for classes with less population. The results have shown that this method can improve the performance of DL algorithms over augmented datasets.

III. CONDITIONAL GENERATIVE ADVERSARIAL NETWORKS

In GAN, there is no control over modes of the data to be generated. The conditional GAN changes that by adding the constraint condition y as an additional parameter to the generator G and hopes that the corresponding images are generated. For example, in MNIST, the digit generated by GAN may be either any digit from 0-9 instead of specified

one or always output the same digit. Fig. 1 shows the network structure of CGAN.

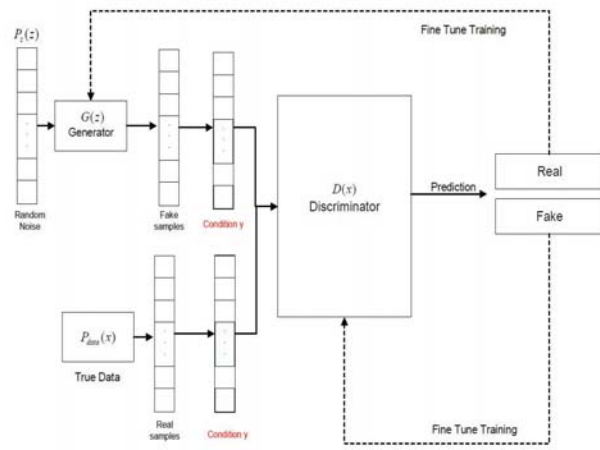


Fig. 1: The network structure of CGAN.

The principle, structure and training process of CGAN are similar to GAN. The cost function is slightly different as is shown in (1):

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)))] \quad (1)$$

As shown in Fig. 1, CGAN's training process includes the following steps:

- Sampling the real data to obtain $P_{data}(x)$, obtaining the label y corresponding to the sampling data $P_{data}(x)$, feed $P_{data}(x)$ and y into the discriminator D , then updating the parameters according to the output results;
- Generating random noise $P_z(z)$, which is then fed into generator G together with label y in the above step, and G generates synthesized data.
- Feeding the synthesized data and the label y generated in the above step into the discriminator D , and G will optimize the parameters according to the output result of D .
- Repeat the above steps until G and D reach the Nash equilibrium.

IV. THE METHODOLOGY OF PACKETCGAN

A. The Workflow of PacketCGAN Based Encrypted Traffic Classification

The workflow of PacketCGAN based encrypted traffic classification is shown in Fig. 2, in which there are three phases: packet data preprocessing, class balancing using PacketCGAN and traffic classifier training.

1) *Phase 1: Generally speaking, packet data preprocessing is the fundamental task of DL modelling. As we all know, the captured network traffic data is often saved in PCAP or PCAPNG format, in which packet bytes of traffic data are saved in hexadecimal. The bytes converted to decimal range*

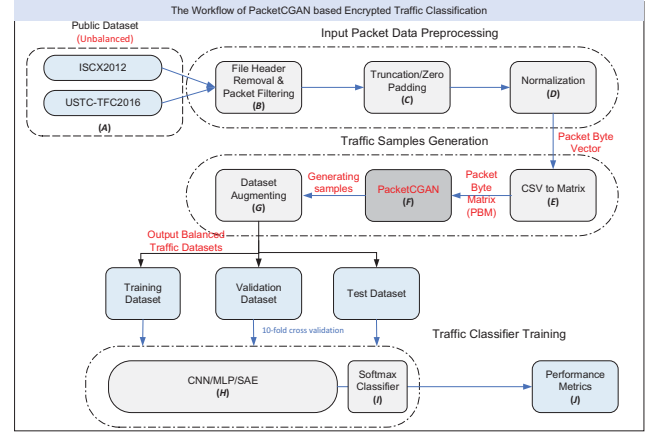


Fig. 2: The workflow of PacketCGAN.

from 0 to 255, which resemble the pixel range of the single-channel grayscale image. Apparently, traffic classification can refer to DL methods of image recognition. However, packets from PCAP files can not directly be used for model training and need to be pre-processed.

Packet data preprocessing consists of three steps as follows:

- File header removal and packet filtering as shown in (B) of Fig. 2. The first 24 bytes of the PCAP file header need to be removed which only contains file information and does not help with traffic classification. Filtering useless packets in PCAP file for classification is also needed, such as ARP and DHCP packets etc., which is irrelevant to specific traffic types.
- Truncation or zero padding as shown in (C) of Fig. 2. During the training of DL model, the input of which needs to be converted into a vector or matrix, which requires the length of the data input to be fixed. Nevertheless, the length of captured packet in dataset tends to be different, so it is necessary to truncate the long packets or padding those short ones based on a fixed and predefined length value [16], which is 1480 in this paper.
- Data normalization as shown in (D) of Fig. 2. Normalizing the pre-processed traffic data to the range [0,1] can improve the accuracy and enhance convergence speed of the model training.

After all above mentioned operations, each processed packet will be formatted to a Packet Byte Vector (PBV) [5] which can be stored in csv format.

2) *Phase 2: Data balancing is the most critical problem in this phase. It is essential to read the vectors of PBV from csv files to build the matrix of PBM as the input of PacketCGAN as shown in (E) of Fig. 2, which will be illustrated in detail in Section. IV-B. Synthesized packet samples will be generated by PacketCGAN in PBM format and then combined with the samples of original dataset to build the new balanced traffic dataset as shown in (E) and (F) of Fig. 2.*

3) *Phase 3: Traffic Classifier Training:* In this paper, we adopted some DL traffic classifiers to evaluate the performance of datasets augmented by PacketCGAN. The new balanced

traffic datasets will always be divided into three parts: training, testing and validation datasets. Finally, classifier related performance metrics will be evaluated to indirectly verify the quality of datasets generated by different data augmentation methods as shown in (H,I,J) of Fig. 2.

B. The Model Architecture and Algorithm Description of PacketCGAN

1) *The Description of Model Architecture:* The model of our proposed PacketCGAN is shown in Fig. 3, in which both G and D adopted MLP as the basic architecture. Random noise $P_z(z)$ as the input of model with Normal Distribution are $N \times 100$ -dimensional vectors, which will be fed into G with $G(z)$ distribution. Different from vanilla GAN, the label of traffic/applications types with one-hot encoding will also be fed into our PacketCGAN as conditional c . The synthesized samples as the output of G are $N \times 1480$ vectors, which will be composed of the matrix of PBM and fed into D with $D(x)$ distribution eventually. Meanwhile, PBM from original unbalanced datasets with $P_{data}(x)$ distribution will be fed into D with the same 1480-dimensionality to compete to reach Nash equilibrium with fine-tune training.

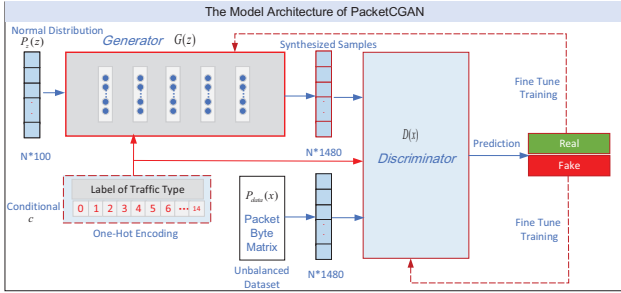


Fig. 3: The Architecture of PacketCGAN.

2) *The Description of Model Algorithm:* In general, PacketCGAN's training mainly includes the following three steps: the training of discriminator D , generator G and fine-tune optimization. During the training of D in step 1, one needs to fix the parameters of G while training D . We adopt the simple MLP to build the D model, in which there are three layers: the input layer, a hidden layer and the output layer. The data for the input layer is derived from the $N \times 1480$ matrix of PBM aforementioned in Section IV-A1 with the distribution of $P_{data}(x)$. The hidden layer consists of 128 neurons with the notation of D_{h1} .

$$D_{h1} = \text{ReLU}(\text{input}_D \cdot W_1^D + b_1^D) \quad (2)$$

The input_D in (2) is composed of the real data x with $P_{data}(x)$ and the traffic/applications types label y in one-hot encoding, such as 0 for SSH, 1 for YouTube. The hidden layer D_{h1} adopts ReLU as the activation function. The output layer has only one neuron referring to real or fake sample as following Equation: 3:

$$D_{out} = D_{h1} \cdot W_2^D + b_2^D \quad (3)$$

The model uses the Adam optimization algorithm as the optimizer to update weights W_1^D , W_2^D and bias b_1^D , b_2^D . The overall training process of the discriminator is shown in Algorithm 1, in which $X_\tau = \begin{pmatrix} x_{11} & \cdots & x_{1k} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mk} \end{pmatrix}$, $y_\tau = \begin{pmatrix} y_{11} \\ \vdots \\ y_{m1} \end{pmatrix}$, k is 1480, which is the data dimension. m is mini_batch referring to a group of packets from dataset.

Algorithm 1 The training of Discriminator of PacketCGAN

Input: real data X_τ and label y_τ from Section IV-B2, τ is the number of iterations
Output: the probability of the output data being real or false
1: Initializing the relevant parameters, e represents the training cycle: epochs.
2: **for** τ in e **do**
3: **for** each batch of m input data **do**
4: concatenate X_τ with y_τ ;
5: Compute the output using Equation. (2);
6: Compute the output using Equation. (3);
7: Output discriminating results according to Equation. (3);
8: Optimize the loss function
9: Update weights and bias;
10: **end for**
11: **end for**

Step 2 is to train the generator G . The parameters of D need to be fixed like step 1. The input layer of G consists of 115 neurons, which includes stochastic noise $P_z(z)$ with 100 neurons and sample label y with 15 neurons of applications types. The hidden layer also has 128 neurons to update the weight and bias of the input data:

$$G_{h1} = \text{ReLU}(\text{input}_G \cdot W_1^G + b_1^G) \quad (4)$$

The 'input' in (5) is made up of random noise $P_z(z)$ and the sample label y with ReLU as the activation function. The output layer contains 1480 neurons with the same dimension of the $P_{data}(x)$. The computed result will be activated by the sigmoid function for discrimination of real or fake.

$$G_{out} = \text{sigmoid}(G_{h1} \cdot W_2^D + b_2^D) \quad (5)$$

The optimization function is the same as the discriminator. The generator training algorithm is shown in Algorithm 2. The sample label y in Algorithm. 2 should be consistent with

$$y \text{ in Algorithm. 1. } Z_\tau = \begin{pmatrix} z_{11} & \cdots & z_{1n} \\ \vdots & \ddots & \vdots \\ z_{m1} & \cdots & z_{mn} \end{pmatrix}, n = 100.$$

Algorithm 2 The training of Generator of PacketCGAN

Input: random noise Z_τ , and label y_τ , τ is the number of iterations

Output: synthesized data g_τ

```

1: Set the relevant initialization parameters,  $e$  represents the
   training cycle: epoches.
2: for  $\tau$  in  $e$  do
3:   for each batch of  $m$  input data do
4:     concatenate  $Z_\tau$  with  $y_\tau$ ;
5:     Compute the output using Equation. (4);
6:     Compute the output using Equation. (5);
7:     Output generation data according to Equation. (5);
8:     Optimal the loss function
9:     Update weights and bias;
10:  end for
11: end for

```

V. EVALUATION AND EXPERIMENTAL RESULTS

A. Experimental Settings

1) *Dataset for Evaluation:* Two public datasets used in this paper are ISCX2012 [17] and USTC-TFC2016 [18], which are both composed of PCAP files. The full name of 'ISCX2012' dataset is 'ISCX VPNnonVPN traffic dataset', which contains many encryption applications or protocols such as HTTPS, SFTP, Facebook, Hangouts, etc. In addition to these applications encapsulated in regular session, some VPN tunnel's applications were also captured in this dataset. 15 applications were chosen to build the original dataset. Apparently, the original chosen dataset happens to the problem of class imbalance, in which the majority class such as Netflix accounting for 25.13%, while the minor class ICQ account for only 2.05%. 10 applications were chosen from USTC-TFC2016. Balanced dataset made by different generative methods are all 10000 samples per applications.

2) *Configurations of the Computing Platform:* The experimental environmental parameters of this paper are shown in Table I. The performance evaluations are conducted using a Dell R730 server with an Intel I7-7600U CPU 2.8 GHz, 16 GB RAM and an external GPU (Nvidia GeForce GTX 1050TI). The software platform for deep learning is built on Keras library with Tensorflow (GPU-based version 1.13.1) as the back-end support.

3) *Description of deep learning based network traffic classifier:* To verify the feasibility and evaluate the performance of the PacketCGAN algorithm, we adopted three classical DL models to classify the traffic over the datasets synthesized by different generative methods, which is MLP, CNN and SAE, respectively. The detail of these three models can be find in our previous works [5].

4) *Performance Metrics for Classification:* The performance metrics used for evaluations of network traffic classifiers are *Precision*, *Recall* and F_1 score.

- **Precision:** precision r_p is the ratio of true positives n_T^P over the sum of n_T^P and false positives n_F^P . In the pro-

TABLE I: Experimental Environment Parameters

Category	Parameters
GPU	Nvidia GPU(GeForce GTX 1050Ti)
Operating System	Win 10
Deep learning platform	TensorFlow 1.13.1 + Keras 1.0.7
CUDA Version	9.0
CuDNN Version	7.6.0

posed classification methods, precision is the percentage of packets that are properly attributed to the targeted application.

$$r_p = \frac{n_T^P}{n_T^P + n_F^P}. \quad (6)$$

- **Recall:** recall r_c is the ratio of n_T^P over the sum of n_T^P and false negatives n_F^N or the percentage of packets in an application class that are correctly identified.

$$r_c = \frac{n_T^P}{n_T^P + n_F^N}. \quad (7)$$

- **F_1 -score:** the F_1 score r_f is a widely-used metric in information retrieval and classification that considers both precision and recall as follows:

$$r_f = \frac{2r_p \cdot r_c}{r_p + r_c}. \quad (8)$$

B. Data Augmenting using PacketCGAN

1) *Data Augmenting Methods handling the problem of class imbalance:* In this paper, four methods were used to address the problem of the unbalanced dataset for the purpose of comparison. The first one is random oversampling method (ROS) mentioned in Section II-B. The essence of this method is to randomly copy some samples of minor class to supplement the dataset. Since the simple method is only a copy of the original data, it will lead to some wrong features be learned by the model easily and even over-fitting. The second method is SMOTE in which minority samples are generated by synthetic samples rather than copies. The third one is vanilla GAN with a generator and discriminator based on MLP. The last method is to generate minor sample data using our PacketCGAN generator. The dataset balanced by different methods is described in V-A1.

2) *The Architecture of PacketCGAN:* In our experiments, a fully connected MLP network was adopted to design generators and discriminators. The generator's input is a 100-dimensional vector generated by random Gaussian noise with additional 15 applications types for ISCX2012 or 10 for USTC-TFC2016 datasets. The next three hidden layers have 128, 256, 512 neurons, and 1480 output neurons with additional 15 for ISCX2012 or 10 for USTC-TFC2016 datasets. Therefore, input of discriminator network is 1495 or 1490 dimension vectors mixed by real traffic data or generated data.

Three hidden layers and output layers are both activated by LeakyReLU.

3) *The Training of PacketCGAN*: Table II shows the parameters of the optimizer, loss function, epoches, and mini_batch used in the deep training model training process.

TABLE II: Training Parameters of Packet CGAN

Training Parameters	Generator	Discriminator
Optimizer	Adma	Adma
Loss Function	Cross-Entropy	Cross-Entropy
Epoches	200000	200000
Mini_batch	64	64

Figure 4 shows the trend of loss of generator and discriminator during the PacketCGAN training process. It can be seen that PacketCGAN does not improve the unstability characteristics of GAN. The loss of G and D always fluctuates within a range instead of convergence. Nevertheless, it is a big improvement that only one CGAN model needs to be trained when using PacketCGAN compared with vanilla GAN. One can generate any samples with the additional input of the applications types as conditional c , while vanilla GAN can only generate one type of samples at a time, that means we can control the output of the PacketCGAN generator so that we can easily generate specified minor classes samples needed to be augmented to handle the problem of class balance.

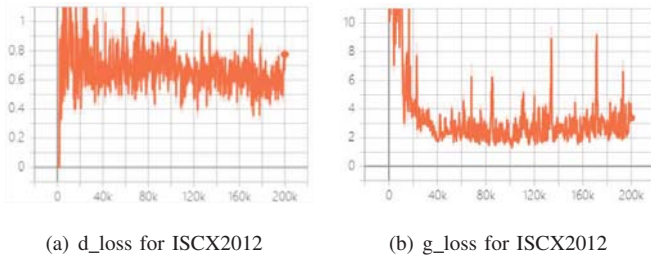


Fig. 4: Loss of PacketCGAN

Fig. 5 shows the Confusion Matrices of the CNN-based encrypted traffic identification model based on 4 datasets augmented by four different generative methods, which are ROS/SMOTE/GAN/CGAN over USTC-TFC2016, respectively. The elements on the diagonal of Confusion Matrix refer to the correct ones of classification and all the others are misclassified. It can be clearly seen that the false positive rate of minor class in Fig. 5(d) is lower than ROS as shown in Fig. 5(a), SMOTE as shown in Fig. 5(b) and GAN as shown in Fig. 5(c).

Fig 6 shows the performance metrics of 4 datasets augmented on ISCX2012 more clearly. As we can see from Fig 6 that PacketCGAN greatly outperforms the other three methods for the minor class like gmail, facebook.

TABLE III: Classification Performance Statistics.

data augmenting methods	Accuracy	Precision	Recall	F1-Score
unbalanced dataset	0.9797	0.9759	0.9775	0.9766
ROS dataset	0.9889	0.9892	0.9889	0.9891
SMOTE dataset	0.9769	0.9751	0.9789	0.971
GAN dataset	0.9766	0.9766	0.9767	0.9766
PacketCGAN dataset	0.9951	0.9936	0.9958	0.9947

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed a CGAN-based traffic data augmenting method called PacketCGAN to solve the class imbalance problem in the dataset. After the specified sample data is generated by the generator of PacketCGAN, the synthesized data is combined with the original data to build a new balanced dataset. We use MLP/CNN/SAE models to verify the classification performance over different datasets. The experimental results show that the balanced dataset augmented by FlowCGAN can achieve better performance than the others including ROS, SMOTE and GAN. In the future, we will further study other types of GAN applications in traffic classification and try to solve the problems of unstable training and mode collapse.

ACKNOWLEDGMENT

This work was supported by the National Science Foundation of China (61972211)

REFERENCE

- [1] P. Wang, F. Ye, and X. Chen, "A smart home gateway platform for data collection and awareness," *IEEE Communications Magazine*, vol. 56, no. 9, pp. 87–93, Sep. 2018.
- [2] P. Wang, X. Chen, F. Ye, and Z. Sun, "A smart automated signature extraction scheme for mobile phone number in human-centered smart home systems," *IEEE Access*, vol. 6, pp. 30 483–30 490, 2018.
- [3] A. Dainotti, A. Pescapè, and K. C. Claffy, "Issues and future directions in traffic classification," *IEEE Network*, vol. 26, no. 1, pp. 35–40, January 2012.
- [4] G. Aceto, D. Ciunzio, A. Montieri, and A. Pescapè, "Mobile encrypted traffic classification using deep learning," in *2018 Network Traffic Measurement and Analysis Conference (TMA)*, June 2018, pp. 1–8.
- [5] P. Wang, F. Ye, X. Chen, and Y. Qian, "Datanet: Deep learning based encrypted network traffic classification in sdn home gateway," *IEEE Access*, vol. 6, pp. 55 380–55 391, 2018.
- [6] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*, pp. 43–48, 2017.
- [7] Z. Wang, "The application of deep learning on traffic identification," Available from <http://www.blackhat.com>, 2015.

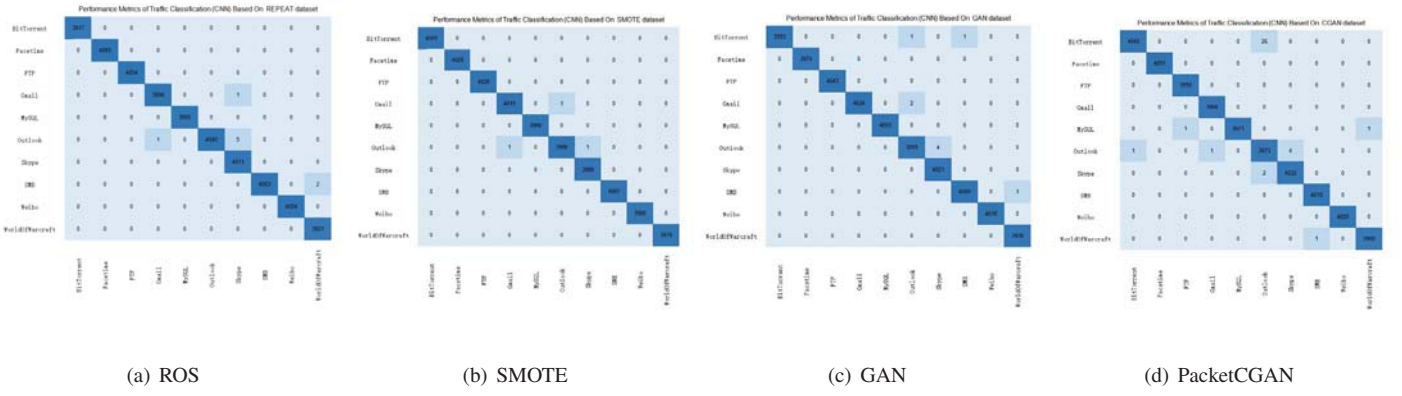


Fig. 5: Confusion Matrices of CNN-based encrypted traffic identification method on USTC-TFC2016

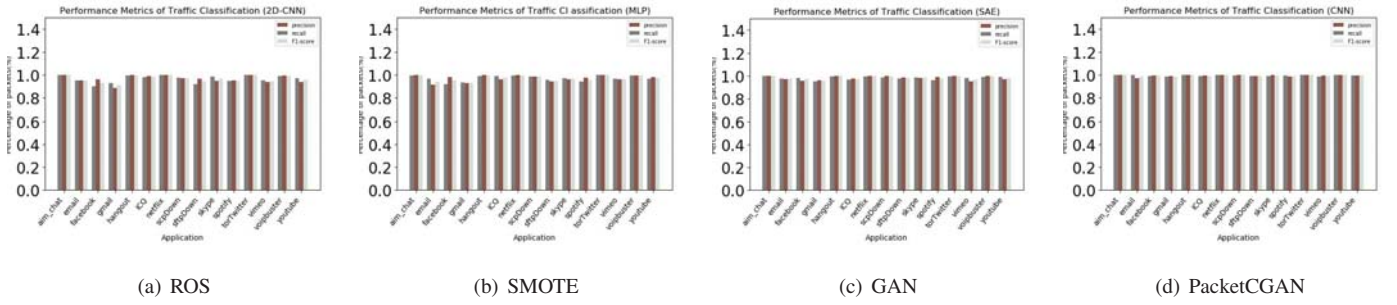


Fig. 6: Performance metrics of DL-based encrypted traffic identification method on ISCX2012

- [8] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for internet of things," *IEEE Access*, vol. 5, pp. 18 042–18 050, 2017.
- [9] L. Vu, C. T. Bui, and Q. U. Nguyen, "A deep learning based method for handling imbalanced problem in network traffic classification," in *Proceedings of the Eighth International Symposium on Information and Communication Technology*, ser. SoICT 2017. New York, NY, USA: ACM, 2017, pp. 333–339. [Online]. Available: <http://doi.acm.org/10.1145/3155133.3155175>
- [10] X. Chen, J. Yu, F. Ye, and P. Wang, "A hierarchical approach to encrypted data packet classification in smart home gateways," in *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)*, Aug 2018, pp. 41–45.
- [11] L. Vu, D. Van Tra, and Q. U. Nguyen, "Learning from imbalanced data for encrypted traffic identification problem," in *Proceedings of the Seventh Symposium on Information and Communication Technology*, ser. SoICT '16. New York, NY, USA: ACM, 2016, pp. 147–152. [Online]. Available: <http://doi.acm.org/10.1145/3011077.3011132>
- [12] N. Japkowicz, "Learning from imbalanced data sets: A comparison of various strategies." AAAI Press, 2000, pp. 10–15.
- [13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *J. Artif. Int. Res.*, vol. 16, no. 1, pp. 321–357, Jun. 2002. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1622407.1622416>
- [14] L. Vu, C. Thanh Bui, and U. Nguyen, "A deep learning based method for handling imbalanced problem in network traffic classification," 12 2017, pp. 333–339.
- [15] R. Hasibi, M. Shokri, and M. Dehghan, "Augmentation scheme for dealing with imbalanced network traffic classification using deep learning," *ArXiv*, vol. abs/1901.00204, 2019.
- [16] C. X. Wang Pan, "Encrypted traffic identification method based on stacked automatic encoder," 2018, pp. 1–8.
- [17] A. Habibi Lashkari, G. Draper Gil, M. Mamun, and A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related features," 02 2016.
- [18] and, , and and, "Malware traffic classification using convolutional neural network for representation learning," in *2017 International Conference on Information Networking (ICOIN)*, Jan 2017, pp. 712–717.